



3.3. N-Queens Problem

"Precision Placement: N Queens, N Solutions"

Main Section:

3. Backtracking

Contents:

[1. Problem Definition](#)

[2. Workflow](#)

[3. Permutation](#)

[Summary](#)

[References](#)

Some problems in Computer Science are famous not because they are practical, but because they perfectly demonstrate how an algorithm thinks. And the N-Queen Problem is one of those problems.

Originally, this problem was introduced as the **Eight-Queens Problem** during the 19th century. The challenge sounded deceptively simple: place eight queens on a chessboard so that none of them attack each other. Over time, however, the idea was generalized into what we now call the **N-Queens Problem**, where the board size becomes $N \times N$, and the number of queens also becomes (N).

Now, if you do not know chess, that is completely fine. You only need to understand one thing about the queen piece: it is extremely powerful. A queen can attack horizontally, vertically, and diagonally. Which means placing even a single queen carelessly can instantly create conflicts everywhere on the board. And that is exactly what makes this problem so interesting.

At first glance, the solution seems easy. Just place queens one by one until the board is full, right? But the moment you try it manually, you quickly realize something important: one bad decision early on can destroy the entire configuration later. This is where **Backtracking** shines.

The N-Queen Problem is considered one of the classic demonstrations of backtracking because the algorithm behaves almost like a human solving a puzzle:

- Try a position → Check whether it is valid → Continue if it works → Go backward if it fails → Again and again, until a complete solution is found.

So although the problem originates from chess, the real lesson here is not about the game itself. It is about learning how algorithms explore possibilities, detect conflicts, abandon bad paths, and systematically search for a valid solution.

And in many ways, this problem becomes the perfect introduction to the true mindset behind backtracking.

1. Problem Definition

The N-Queen Problem asks a very specific question:

Can we place N queens on an $N \times N$ chessboard such that no two queens attack each other?

Since queens can move horizontally, vertically, and diagonally, this means no two queens may share the same row, the same column, or the same diagonal

For the classical version where $N = 8$, the problem becomes the famous **Eight-Queens Problem**. The objective is to place eight queens on an 8×8 board while satisfying all attack constraints simultaneously. At first, this sounds manageable. But the difficulty grows extremely quickly as N increases. Every new queen reduces the number of safe positions available for the next queens, creating an enormous search space of possible configurations.

What makes the problem fascinating is that there is no simple mathematical formula that directly generates all solutions for arbitrary N . Even today, the exact number of solutions is only completely known for relatively small board sizes. For the classical case, there are exactly 92 valid solutions when $N = 8$.

Interestingly, solutions exist for almost every natural number N , except: $N = 2$ and $N = 3$, where constructing a valid arrangement is impossible.

So the challenge of the N-Queen Problem is not simply placing queens randomly. It is about finding a configuration where every queen coexists without conflict — a perfect example of constraint satisfaction through systematic search.

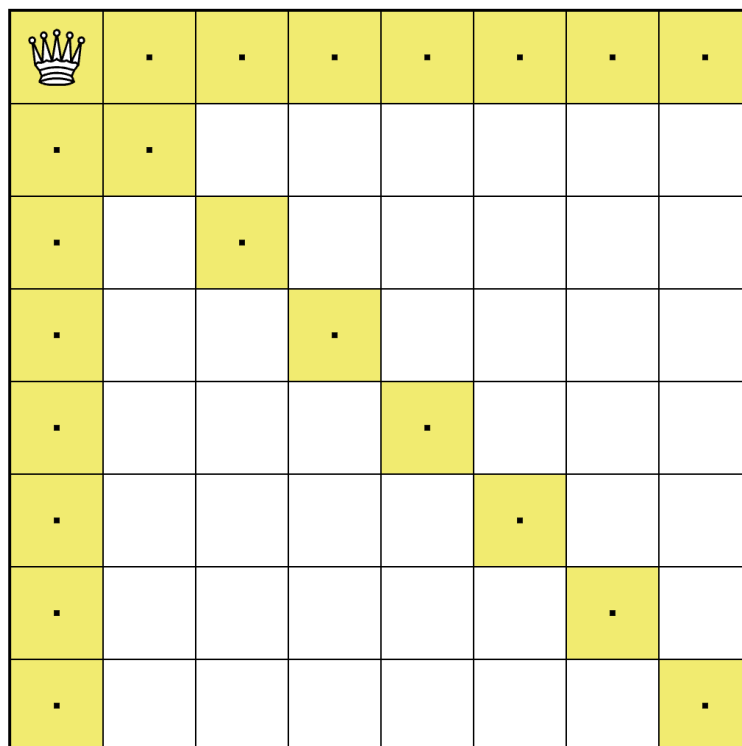
2. Workflow

Now let's finally see how Backtracking actually solves the N-Queen Problem.

The central idea is surprisingly human-like. We place queens one by one onto the board while constantly checking whether the current arrangement is still valid. If at some point a queen creates a conflict, we immediately abandon that path and go backward to try another possibility.

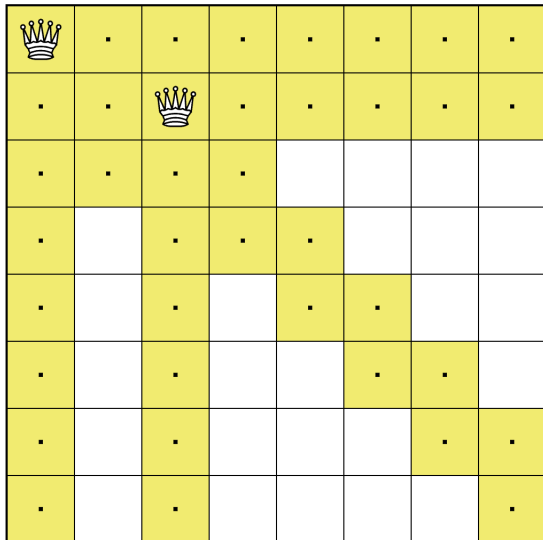
That "going backward" process is exactly what we call **Backtracking**. The algorithm usually works row by row (or column by column depending on implementation). For simplicity, let us think in terms of rows.

- We begin with the first row and place a queen in the first available column.

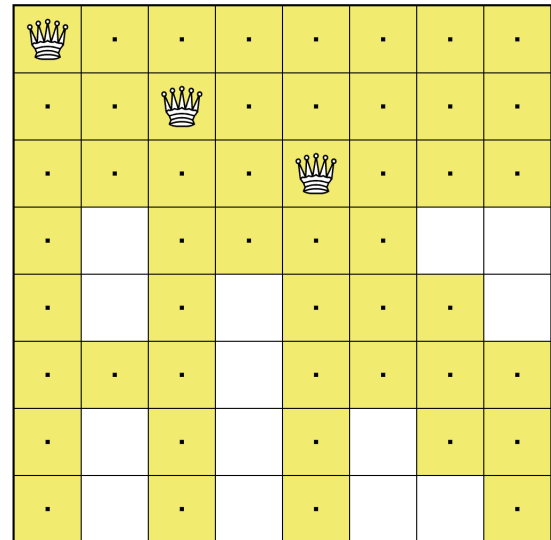


1 / 8 Queens Placed

Then we move to the next row and search for a safe position: not in the same column, not on the same diagonal, and not attacking any previous queen

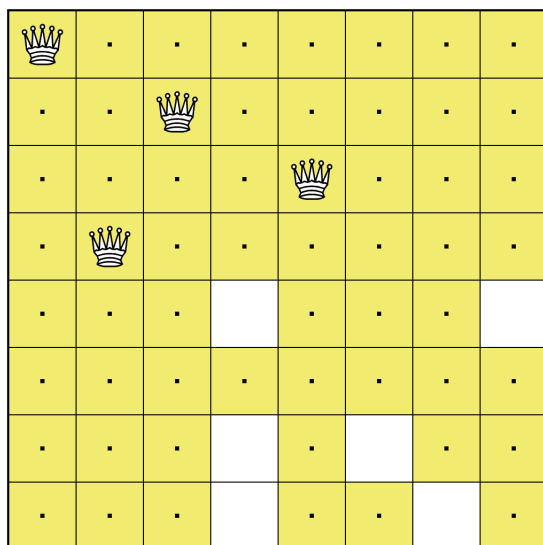


2 / 8 Queens Placed

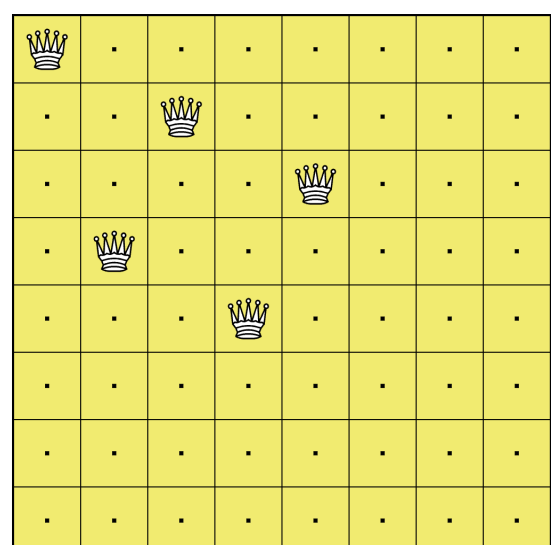


3 / 8 Queens Placed

If a safe position exists, we place the queen and continue forward.



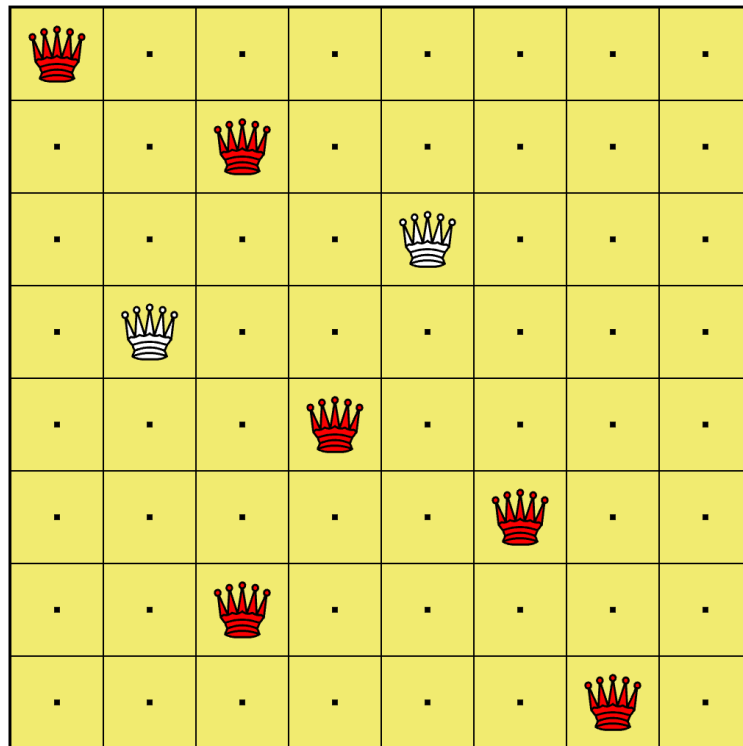
4 / 8 Queens Placed



5 / 8 Queens Placed

But eventually, something important happens. A row may appear where no valid position exists anymore. Every possible square is under attack from previously

placed queens.



8 / 8 Queens Placed

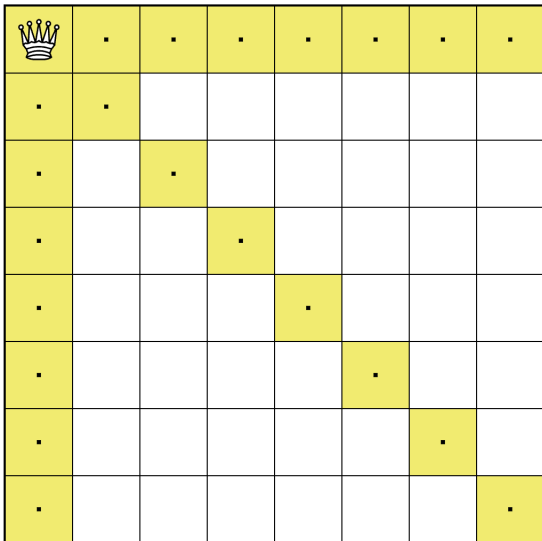
All 8 Queens are placed. But this is an invalid solution

When that happens, the algorithm realizes: ***"One of my earlier decisions must have been wrong."***

So instead of restarting everything from the beginning, it simply removes the most recently placed queen and tries the next possible position in that previous row.

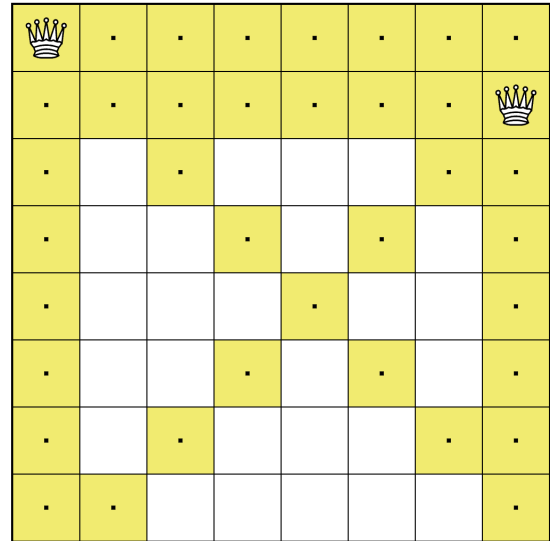
This creates the classic Backtracking cycle: ***Place a queen*** → ***Check validity*** → ***Continue if valid*** → ***Go backward if impossible***. Again and again, until a complete configuration is discovered.

For example, the process may look something like this:



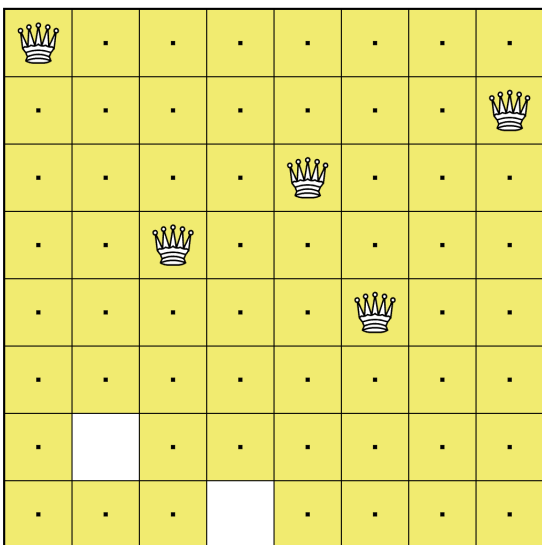
1 / 8 Queens Placed

Place a queen in Row 1



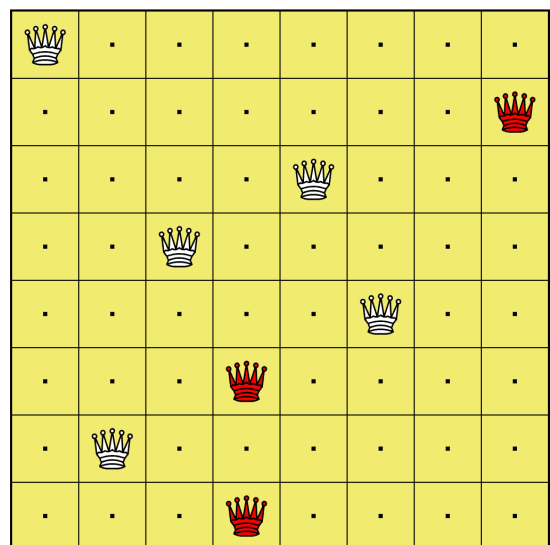
2 / 8 Queens Placed

Move to Row 2 and place another queen safely



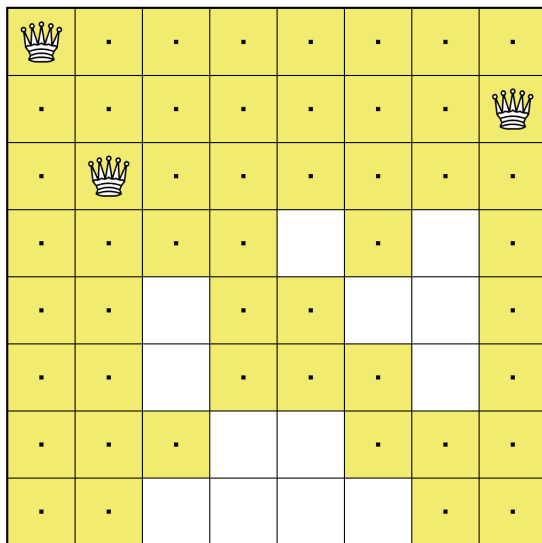
5 / 8 Queens Placed

Continue filling Row 3, Row 4, and so on



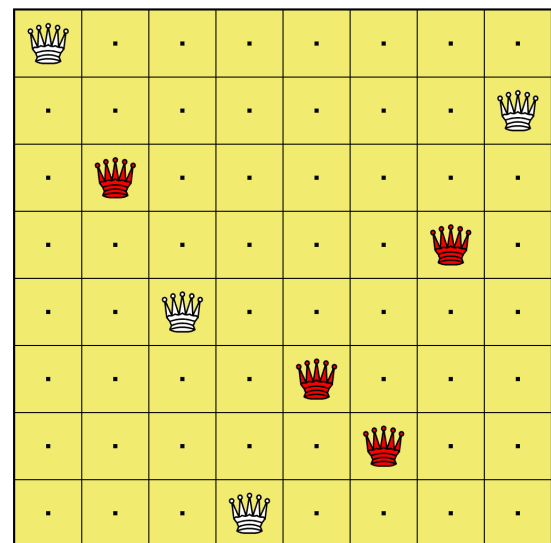
8 / 8 Queens Placed

Suddenly discover Row 6 has no valid position



3 / 8 Queens Placed

Remove the queen from Row 4, 5 and 6.
Backtrack all the way to Row 3 and try
another cell



8 / 8 Queens Placed

Try another possible cell in Row 4 and so
on. Continue searching again

The algorithm systematically explores every possible arrangement while rejecting invalid states as early as possible. And eventually, when all eight queens are successfully placed: 8 queens placed without conflict → A valid solution is found and recorded.

After recording the solution, the algorithm does not stop immediately. It continues backtracking to search for other possible solutions as well. This process continues until every possible configuration has been explored.

So although Backtracking feels intelligent, it is still fundamentally a brute-force search technique. The difference is that it avoids wasting time exploring obviously impossible paths. Instead of blindly generating every board arrangement, it prunes invalid configurations early. And that pruning is what makes Backtracking dramatically more efficient than naive brute force.

Below is the simple Python implementation of 8-Queens Problem:

```
def is_safe(board, row, col):
    # Row on the left
    for i in range(col):
        if board[row][i] == 1:
            return False
```

```

# Upper Diagonal on the left
for i, j in zip(range(row, -1, -1), range(col, -1, -1)):
    if board[i][j] == 1:
        return False
# Under Diagonal on the left
for i, j in zip(range(row, N, -1), range(col, -1, -1)):
    if board[i][j] == 1:
        return False

return True

```

```

def queen_solution(board, col):
    # Case 1: All queens
    if col >= N:
        return True
    # Case 2: Queen can be put in each row
    for i in range(N):
        if is_safe(board, i, col):
            board[i][col] = 1
            if queen_solution(board, col + 1):
                return True
            board[i][col] = 0
    # If a queen cannot be put anywhere
    return False

```

```
# 8-Queen using Backtracking
import numpy as np

N = 8 # Table 8x8
board = np.zeros((N, N), int)
if queen_solution(board, 0) is False:
    print("Unsolvable")

print("Result:")
for i in range(N):
    for j in range(N):
        print(board[i][j], end=" ")
    print()
```

Result:

```
1 0 0 0 0 0 0 0
0 0 0 0 0 0 1 0
0 1 0 0 0 0 0 0
0 0 0 0 0 1 0 0
0 0 0 0 0 0 0 1
0 0 1 0 0 0 0 0
0 0 0 0 1 0 0 0
0 0 0 1 0 0 0 0
```

3. Permutation

The N-Queens Problem also reveals another important idea closely connected to Backtracking: **Permutation**.

If we carefully observe the rules of the problem, we notice something interesting. Since no two queens may share the same row or column: every row must contain exactly one queen, and every column must also contain exactly one queen. That means the problem can actually be represented using permutations.

Instead of storing an entire chessboard, we can simply store the column position of each queen. For example: [3, 1, 4, 2, 5, 7, 6, 8] represents:

- Queen in Row 1 → Column 3
- Queen in Row 2 → Column 1

- Queen in Row 3 → Column 4
- and so on

Notice what happened here each number is unique, no columns repeat, therefore column conflicts are automatically eliminated

This simplifies the problem significantly because we no longer need to check row or column duplication. The only remaining conflict becomes the diagonals.

So the idea becomes:

1. Generate a permutation of numbers from 1 to 8
2. Place queens according to that order
3. Check whether diagonal conflicts exist
4. If conflicts occur, try another permutation

Eventually, one of the permutations will produce a valid arrangement.

However, there is still a major issue. The total number of permutations grows extremely fast. For the eight-queens problem: $8! = 40320$ possible permutations exist. That means the algorithm may need to examine tens of thousands of arrangements just for $N = 8$.

And for larger values of N , the number becomes enormous. So while the permutation method reduces complexity conceptually, it still remains computationally expensive because it essentially performs large-scale trial and error.

Backtracking improves this process further by rejecting invalid partial arrangements early, instead of waiting until a full permutation is generated. In other words: permutation generates possibilities, backtracking intelligently filters them. And together, these ideas form one of the classic foundations of combinatorial search problems.

Summary

The N-Queens Problem is one of the most iconic demonstrations of Backtracking because it perfectly illustrates how algorithms explore possibilities while dealing with constraints.

Instead of blindly trying every board configuration, Backtracking builds the solution step by step. Queens are placed one at a time, and whenever a conflict

appears, the algorithm immediately goes backward and searches for another path. This creates an intelligent trial-and-error process that avoids many impossible configurations early.

We also discovered another important concept hidden inside the problem: permutations. Since each queen must occupy a unique row and column, the board can be represented as an ordered arrangement of numbers. This reduces unnecessary checks and reveals a more mathematical way of thinking about the problem.

However, both approaches also reveal an important reality of combinatorial problems: the number of possibilities grows extremely quickly. Even though Backtracking is far more efficient than naive brute force, the N-Queen Problem still becomes computationally expensive for large values of N .

And that is exactly why this problem is so important in Computer Science. It teaches something much deeper than chess. It teaches how algorithms: explore search spaces, handle constraints, abandon impossible decisions, and systematically construct valid solutions

And in many ways, this becomes the true philosophy behind Backtracking itself.

References

<https://www.brainbashers.com/queens.asp>

<https://www.geeksforgeeks.org/dsa/n-queen-problem-backtracking-3/>

<https://favtutor.com/blogs/n-queen-problem>

https://www.youtube.com/watch?v=xFv_HI4B83A